Extreme CI/CD for dbt

Nothing bad leaves the laptop. Quality enforced before a commit exists.

- Project 2 of 3 · A shift-left data-engineering portfolio.
- Runnable dbt + DuckDB architecture.
- No warehouse account required. Synthetic data only.

Two failure modes rot every dbt project

Quality Drift

The Symptom: Models are merged with no tests, no documentation, and inconsistent SQL formatting.

The Result: Six months later, technical debt solidifies. Nobody can safely change anything.

CI Cost Blow-out

The Symptom: Every pull request triggers a full rebuild of the entire data warehouse.

The Result: CI becomes excruciatingly slow and expensive. Engineers learn to route around it and skip testing.

A slow, costly CI pipeline encourages skipped tests, directly exacerbating quality drift.

Shift quality left: To the laptop and to a smart CI

Catching problems at the cheapest possible intervention point.

1. The Laptop (Pre-commit)

Mechanism: Blocks non-compliant SQL mechanically.

Intervention: *Before the commit exists.*

2. The Pipeline (Slim CI)

Mechanism: Builds only changed models and their downstream children.

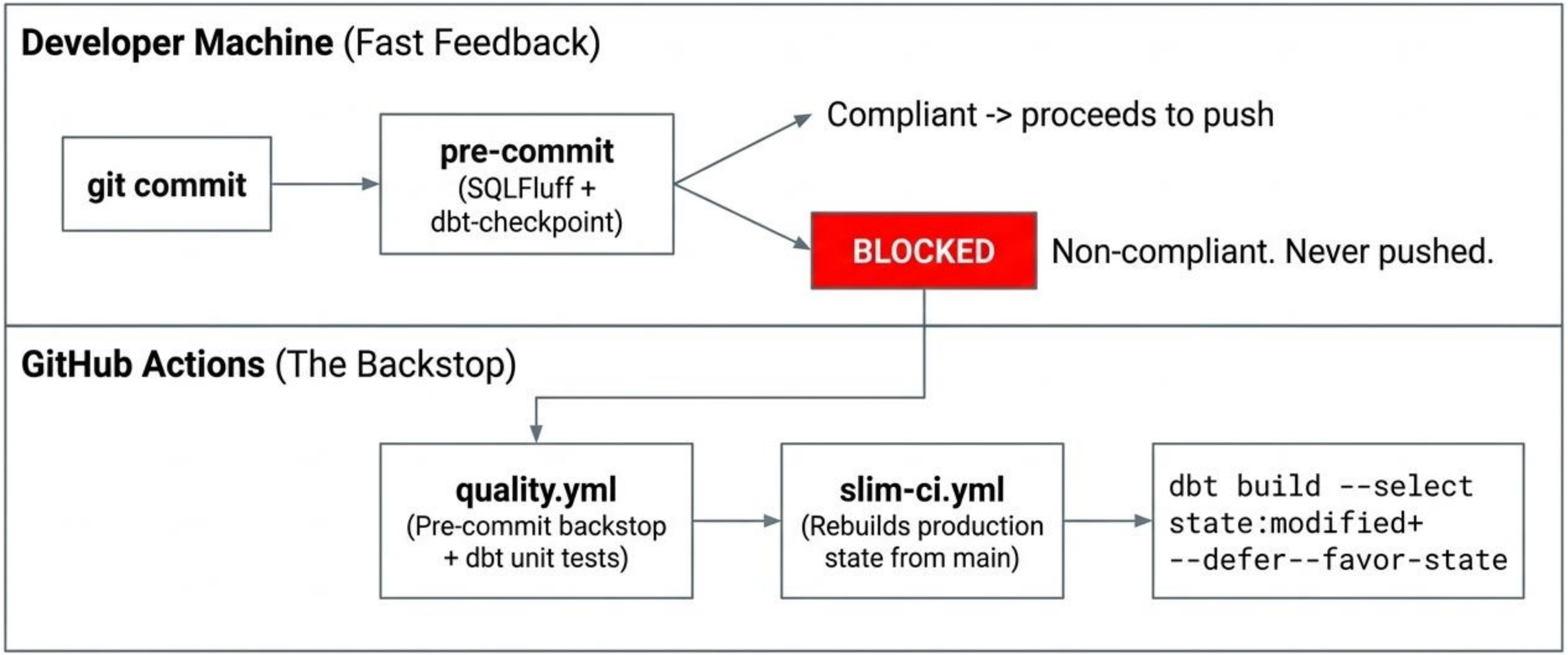
Intervention: *During the Pull Request.*

3. Correctness (Unit Tests)

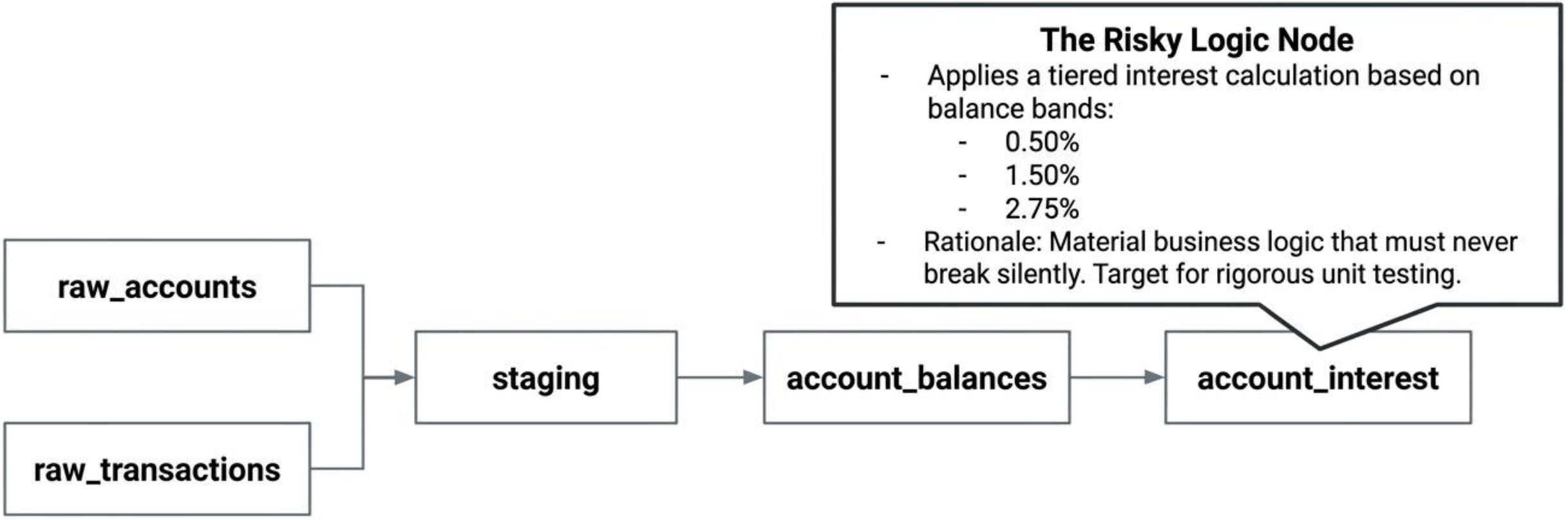
Mechanism: Validates material business logic against mock inputs without warehouse round-trips.

Intervention: *Zero-flakiness logical verification.*

Architectural Blueprint: Local enforcement to stateful CI



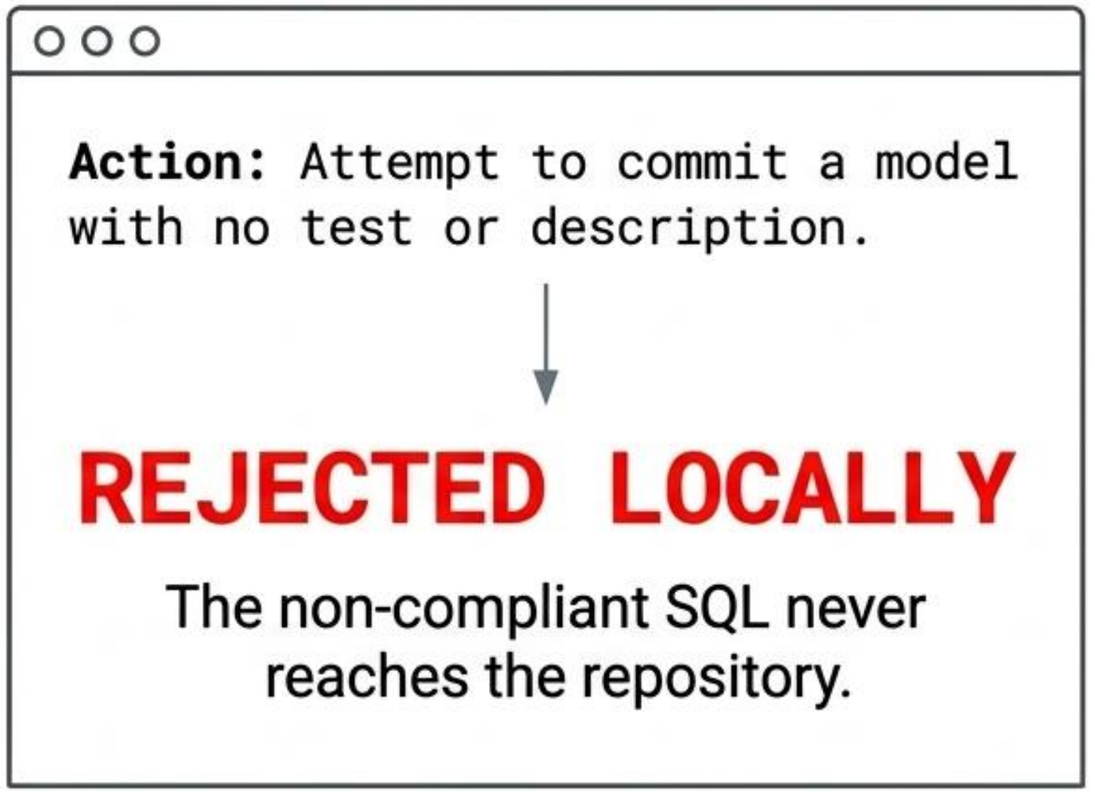
The Data Model: A small, realistic finance lineage



Guard 1: Nothing bad leaves the laptop

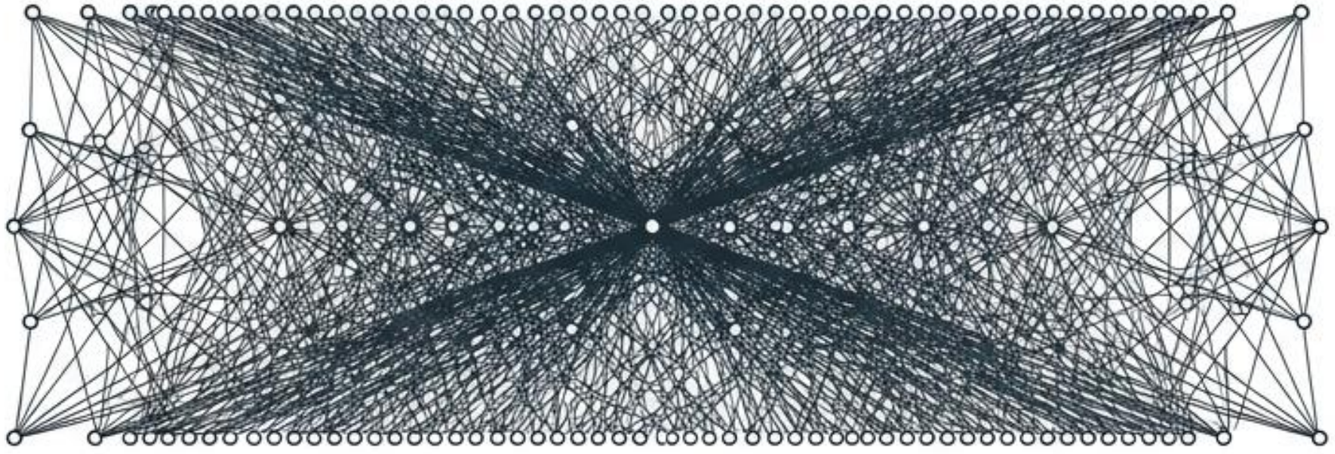
`pre-commit = SQLFluff + dbt-checkpoint`

- **SQLFluff**:
  Lints and auto-formats SQL.
  Coding style is enforced mechanically, eliminating PR arguments.
- **dbt-checkpoint**:
  Validates metadata governance.
  Every model must possess tests, a description, and described columns.



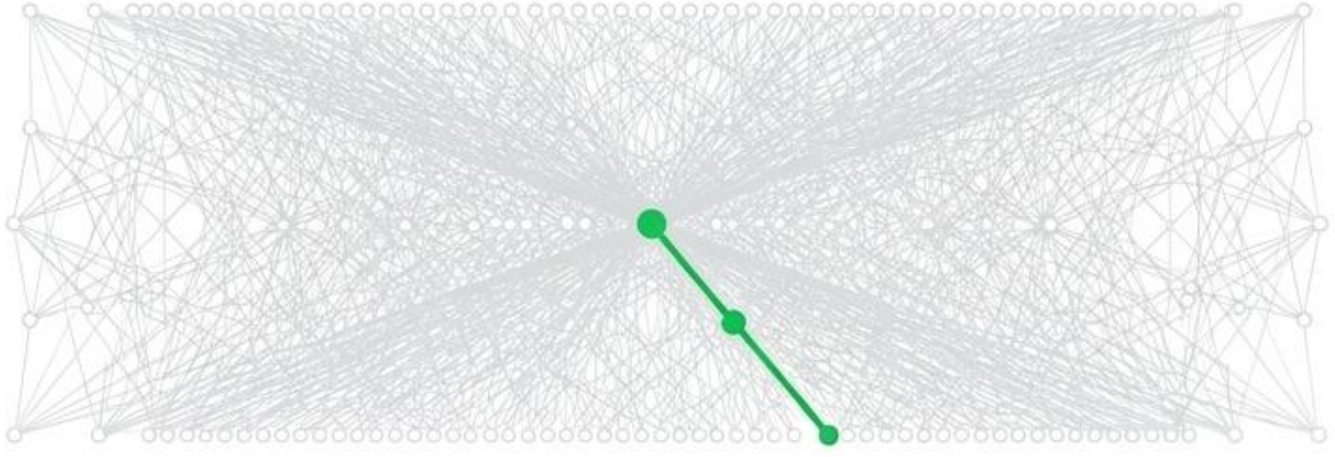
Guard 2: Stateful CI builds only what changed

Slim CI utilizing state:modified+ and deferral.



Traditional CI (Rebuild All)

Change one model -> Rebuild 500 models (Minutes vs. Hours of cloud spend).



Slim CI (Rebuild Changed)

Compare branch to production state. Defer unchanged nodes. Build only the modified model and its children.

Guard 3: Unit-test the risky logic

Schema tests catch nulls; unit tests catch wrong maths.

Schema Tests	dbt Unit Tests
• **Target:** Generic data quality (uniqueness, not-null, accepted values). - **Execution:** Runs against the live warehouse. - **Limitation:** Cannot verify if a complex calculation is mathematically correct.	• **Target:** The tiered-interest calculation. - **Execution:** Validates transformation logic against static mock inputs. Includes exact tier boundaries (e.g., exactly $1,000 and $10,000). - **Advantage:** Zero warehouse round-trips. Zero flakiness. “Given these inputs, assert these outputs.”

The ‘Money Shots’: Demonstrations of prevention

Reproducible proof of the pipeline in action.

Guard 1 Executed	`Model missing description/test. -> git commit REJECTED by pre-commit.`	**Takeaway: Bad code blocked before PR creation.**
Guard 2 Executed	`Modifying one model out of the lineage. -> Slim CI log confirms 1 of 5 models built` **`1 of 5 models built`**	**Takeaway: Surgical rebuilds save compute time and money.**

Engineering decisions & trade-offs

Architectural choices a reviewer should trust.

Decision	Alternative	Rationale
DuckDB	Cloud Warehouse	Patterns (Slim CI, deferral) are warehouse-agnostic. DuckDB ensures the repo is reproducible anywhere in minutes without cloud keys.
Slim CI via Prod Rebuild	Long-lived Warehouse	Honest reconstruction of production state. The selection logic (state:modified+) remains identical whether rebuilding prod or committing a manifest.
Unit-test risky logic only	100% Ritual Coverage	Test where the risk actually resides (calculations) rather than enforcing uniform, low-value generic coverage everywhere.

Regulatory mapping

Change-management discipline that maps to prudential expectations.

APRA CPS 230	**Focus:** Controls over changes to systems supporting critical operations. **Engineering Evidence:** Every Pull Request leaves a durable, timestamped CI record demonstrating effective implementation of controls.
APRA CPG 235	**Focus:** Data quality managed across the lifecycle. **Engineering Evidence:** Pre-commit hooks enforce metadata governance and quality checks before code integration.

General information, fact-checked against the APRA fact base. Not compliance advice.

Honest scope

What was simplified, and why.

Scale

Choice: Small model set and synthetic seeds.

Why: Enough to perfectly demonstrate lineage and selective builds without the noise of a massive production warehouse.

Environment

Choice: ‘Production’ reconstructed from main in CI.

Why: Avoids maintaining a long-lived warehouse. The compute saving is illustrative here, but the exact same mechanism saves real hours at scale.

Complexity

Choice: Single currency, no incremental models or snapshots.

Why: Deliberately kept out of scope to maintain readability of the core CI/CD patterns.

Run it yourself & skills demonstrated

```
make setup && make install-hooks

make demo # seed -> build -> test -> unit tests
make unit-test # logic vs mock inputs
```

(Clone and run locally in ~3 minutes)

- dbt (Data Build Tool)
- Slim CI (state:modified+, deferral)
- dbt Unit Tests
- SQLFluff
- dbt-checkpoint
- Cost-aware CI/CD
- Pre-commit engineering

See the code

Repository: github.com/initiative1972/shift-left-2-dbt-cicd
The .pre-commit-config.yaml and Slim CI workflow are designed to be lifted straight into any dbt repo.

Portfolio Context: Part of a 3-project shift-left data engineering portfolio (Prevent -> Enforce -> Prove).

Author Details:
Henry Yan
AI Engineer & Data Architect · Melbourne

(Personal project · synthetic data only · general information, not advice · views my own)